Student Name: Bhavya **UID: 24BAI70791**
Branch: CSE(AI & ML) **Section/Group: 24AIT_KRG-1/G2**
Semester:4
Subject Name: Database Management System **Subject Code: 24CSH-298**

Experiment – 7

1. Aim of the Session

To design and implement a stored procedure that accepts gender as an input parameter and returns the total number of employees based on the given gender, demonstrating procedural programming and parameterized logic in PL/SQL.

2. Software Requirements

Database Management System:

- Oracle Database Express Edition (Oracle XE)
- PostgreSQL Database

Database Administration Tool / Client Tool:

- Oracle SQL Developer (for Oracle XE)
- pgAdmin (for PostgreSQL)

3. Objectives

- To understand and implement stored procedures in PL/SQL
- To use **IN and OUT parameters** effectively
- To perform **dynamic querying based on user input**
- To apply **aggregate functions (COUNT)** inside procedures

4. Procedure of the Experiment

1. Open database tool and connect to the database.
2. Create the employees table.
3. Insert sample employee records.
4. Verify data using SELECT query.
5. Create a stored procedure with input and output parameters.
6. Use COUNT(*) to count employees based on gender.
7. Compile and save the procedure.
8. Call the procedure by passing gender value.
9. Display the result using output statement.
10. Verify the correctness of the output.

5. Practical / Experiment Steps

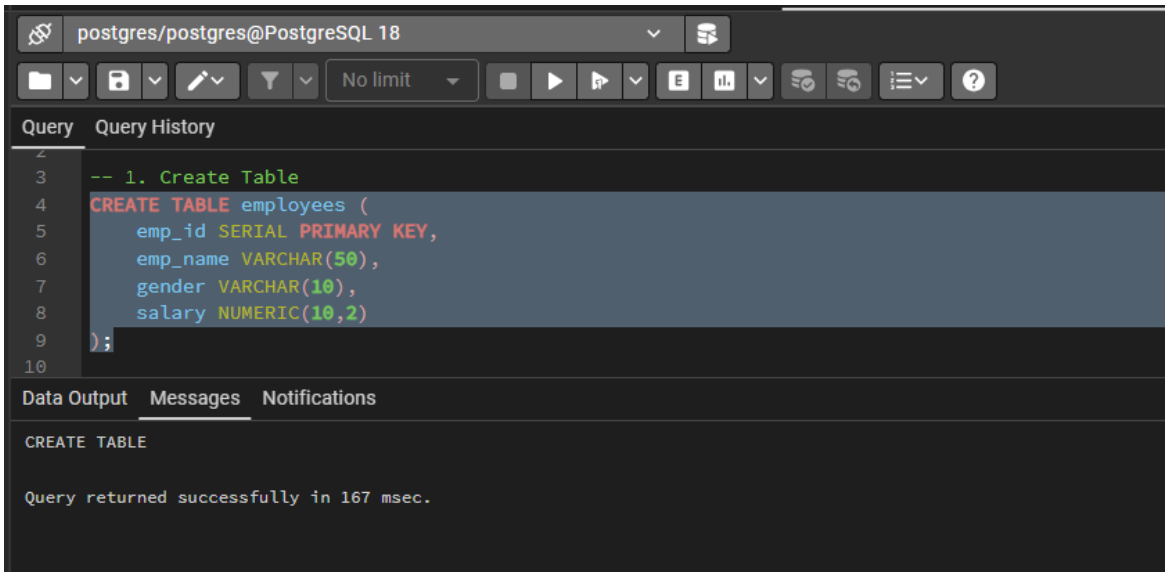
PROGRAM CODE:

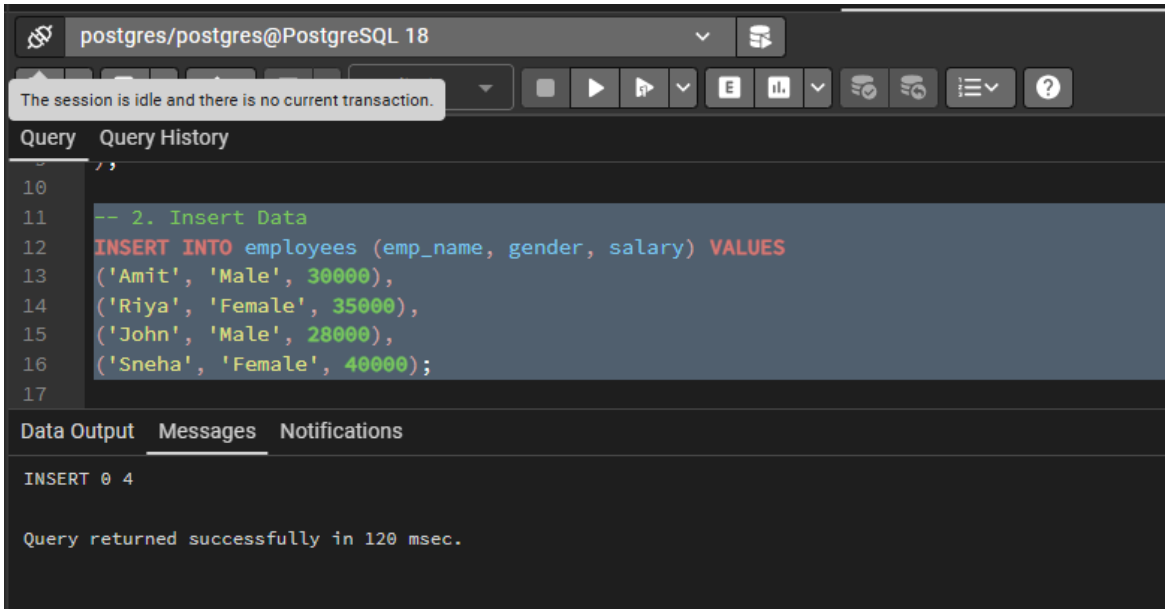
```
-- 1. Create Table
CREATE TABLE employees (
    emp_id SERIAL PRIMARY KEY,
    emp_name VARCHAR(50),
    gender VARCHAR(10),
    salary NUMERIC(10,2)
);


-- 2. Insert Data
INSERT INTO employees (emp_name, gender, salary) VALUES
('Amit', 'Male', 30000),
('Riya', 'Female', 35000),
```

```
('John', 'Male', 28000),
('Sneha', 'Female', 40000);

-- 3. Procedure
CREATE OR REPLACE PROCEDURE get_employee_count_by_gender(
    IN p_gen VARCHAR(20),
    OUT count_emp INT,
    INOUT status VARCHAR
)
AS $$
BEGIN
    SELECT COUNT(*) INTO count_emp
    FROM employees
    WHERE gender = p_gen;

    status := 'SUCCESS';
END;
$$ LANGUAGE plpgsql;

-- 4. Calling Procedure
DO $$
DECLARE
    gen VARCHAR(20) := 'Male';
    count_by_gen INT;
    status VARCHAR(20) := 'Fail';
BEGIN
```

```
CALL get_employee_count_by_gender(gen, count_by_gen, status);


RAISE NOTICE 'GENDER IS % YOUR COUNT IS % AND STATUS IS %',
    gen, count_by_gen, status;
END;
$$;
```

6. Input / Output Details and Screenshot





Query | Query History

```
17
18  -- 3. Procedure
19  CREATE OR REPLACE PROCEDURE get_employee_count_by_gender(
20      IN p_gen VARCHAR(20),
21      OUT count_emp INT,
22      INOUT status VARCHAR
23  )
24  AS $$
25  BEGIN
26      SELECT COUNT(*) INTO count_emp
27      FROM employees
28      WHERE gender = p_gen;
29
30      status := 'SUCCESS';
31  END;
32  $$ LANGUAGE plpgsql;
33
34  -- 4. Calling Procedure
```

Data Output | Messages | Notifications

```
CREATE PROCEDURE

Query returned successfully in 308 msec.
```

Query | Query History

```
33
34  -- 4. Calling Procedure
35  DO $$
36  DECLARE
37      gen VARCHAR(20) := 'Male';
38      count_by_gen INT;
39      status VARCHAR(20) := 'Fail';
40  BEGIN
41      CALL get_employee_count_by_gender(gen, count_by_gen, status);
42
43      RAISE NOTICE 'GENDER IS % YOUR COUNT IS % AND STATUS IS %',
44          gen, count_by_gen, status;
45  END;
46  $$;
```

Data Output | Messages | Notifications

```
NOTICE:  GENDER IS Male YOUR COUNT IS 2 AND STATUS IS SUCCESS
DO

Query returned successfully in 336 msec.
```

7. Learning Outcome

- Understand the concept of stored procedures.
- Learn to use **IN, OUT, and INOUT parameters**.
- Gain knowledge of passing dynamic input values.
- Apply **COUNT() function** for data analysis.
- Develop reusable database logic.
- Understand real-world data handling scenarios.